Smart AI Audio Summarizer

Execution Instructions

This guide describes how to configure and run the React/Vite public web implementation locally. The browser performs AI inference through Transformers.js; Supabase provides authentication and metadata persistence.

1. Software requirements

Install a current Node.js LTS release from the official Node.js website. npm is included with Node.js. Use a current Chrome or Edge browser, and ensure internet access is available for Supabase requests and browser model downloads.

Verify installation in Command Prompt or a terminal:

```
node --version
npm --version
```

2. Open the project folder

Extract the submission ZIP. Open a terminal inside the **source_code** folder, the folder containing **package.json**. Run the remaining commands from this directory.

3. Install dependencies

```
npm install
```

4. Configure Supabase

Copy **.env.example** to **.env.local** in the source_code folder. Then replace the two placeholders with the project URL and publishable key provided by the project owner or Supabase project settings.

```
:: Windows Command Prompt
copy .env.example .env.local

:: PowerShell / macOS / Linux
cp .env.example .env.local
```

Required variables:

```
VITE_SUPABASE_URL=https://YOUR-PROJECT-REF.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=YOUR_SUPABASE_PUBLISHABLE_KEY
```

Do not use a service-role key or other private server secret here. Any variable prefixed with VITE_ is available to client-side code. Supabase Row Level Security (RLS) must be correctly configured for data protection.

5. Run the website

```
npm run dev
```

Open the local URL printed by Vite, normally http://localhost:5173/. Keep the terminal open while using the development server.

6. Create a production build

```
npm run build
npm run preview
```

The production files are generated in the dist folder. dist is generated output and is not required in the source-code ZIP.

7. Main usage workflow

• Open the application and sign in or create an account if authentication is enabled for the Supabase project.

• Upload a recording or use browser recording controls, then start transcription.

• Wait for the browser-compatible Whisper model to load and process the media. First use may download a large model.

• Review the timestamped transcript and use transcript search/navigation where available.

• Generate a summary. If the summarization model fails to load, the implementation includes an extractive fallback.

• Choose German, Spanish, French, or Italian to translate the summary.

• Export the relevant output as a PDF if needed.

8. Troubleshooting

Missing environment variables: Ensure .env.local is beside package.json, variable names are exact, values have no extra quotation marks unless required, and restart the dev server after changes.

Supabase errors: Check that the Supabase project is active, authentication is configured, expected tables exist, and RLS policies allow the current user's intended operations.

Model loading errors: Check internet access, browser console errors, available RAM, and browser compatibility. Retry after refreshing; model downloads can take time.

Microphone access: Allow microphone permission in the browser. Use localhost or a secure HTTPS origin.

PDF export errors: Try a current Chrome/Edge version and inspect the browser console.

9. Project links

Live website: https://smart-ai-audio-summarizer.vercel.app/

GitHub repository: https://github.com/touliktalukder02-afk/Smart-AI-Audio-Summarizer

10. Security and scope notes

The application uses browser-side Transformers.js models for speech recognition, summarization, and translation. Supabase stores account and analysis metadata; the code comments indicate the actual audio remains on the user's device during transcription. The browser must fetch model files as needed. Do not add real secrets to the submission ZIP or public Git history.

This guide describes the source code in the submitted ZIP. Exact behavior can vary with browser support, Supabase schema/policies, and model availability.